

Blockchain Technologies 2 Final Project Report

Option E - Decentralized Insurance Pool

Security Audit Report

Team roles:

Zhambyl Dulat - smart contracts, integration, deployment scripts, final repository;

Murat - frontend integration and subgraph structure;

Daulet - tests, coverage, audit notes, gas report.

Repository: dooolat/decentralized-insurance-pool

Reviewed branch state: develop

Reviewed commit: 9c22d97938c80048911199ecc7737416b3e3664b

This report documents a team-authored internal security review of the course project codebase. It is intentionally honest about what was reviewed, what was not reviewed, and which operational assumptions still remain. It does not claim mainnet readiness, final Slither cleanliness, or a hosted subgraph deployment.

1. Executive Summary

The reviewed repository implements a decentralized insurance pool prototype including an ERC-4626 collateral vault, policy NFT issuance, Chainlink-based claim triggers, DAO governance via Governor plus Timelock, an AMM module, and an isolated UUPS upgradeability example. The codebase shows several strong engineering choices:

- a reserve-aware vault that caps withdrawals against active coverage;
- separation of policy purchase and claim execution paths;
- stale and incomplete Chainlink round rejection in the oracle adapter;
- governance and timelock primitives built on standard OpenZeppelin modules;
- broad automated coverage including unit, fuzz, invariant, and deployment checks in the repository test suite.

The audit did not identify any Critical or High severity findings in the reviewed commit. The largest remaining risks are operational and governance related rather than classic arithmetic or reentrancy bugs. In particular:

- risk thresholds are stored in raw feed units without on-chain decimal normalization safeguards;
- the deployment/verification flow does not yet prove that bootstrap timelock admin authority was revoked after setup;
- the upgradeable example proxy is not covered by the current governance handoff verification checklist.

Finding Summary

Summary of first-party findings:

- Critical: 0
- High: 0
- Medium: 0
- Low: 5
- Informational: 4

Overall conclusion:

- the contract set is technically coherent and well-tested for a course project;
- the core design does not show an obvious direct fund-drain bug in the reviewed commit;
- the remaining priority work is governance hardening, deployment runbook tightening, and replacing the pending Slither appendix with a real rerun.

2. Scope

2.1 Commit and Branch

- branch reviewed: develop
- commit reviewed: 9c22d97938c80048911199ecc7737416b3e3664b

2.2 Files in Scope

Smart contracts:

- `contracts/amm/InsuranceAMM.sol`
- `contracts/benchmark/AssemblyBenchmark.sol`
- `contracts/factory/InsuranceFactory.sol`
- `contracts/governance/ProtocolGovernor.sol`
- `contracts/governance/ProtocolTimelock.sol`
- `contracts/insurance/ClaimManager.sol`
- `contracts/insurance/InsurancePool.sol`
- `contracts/insurance/RiskRegistry.sol`
- `contracts/interfaces/IInsurancePool.sol`
- `contracts/nft/PolicyNFT.sol`
- `contracts/oracle/ChainlinkOracleAdapter.sol`
- `contracts/token/GovernanceToken.sol`
- `contracts/upgrade/UpgradeableInsurancePoolV1.sol`
- `contracts/upgrade/UpgradeableInsurancePoolV2.sol`
- `contracts/vault/InsuranceVault.sol`

Deployment and verification scripts:

- `script/Deploy.s.sol`
- `script/VerifyDeployment.s.sol`
- `script/UpgradeV2.s.sol`

Supporting evidence considered during review:

- repository tests under `test/`
- deployment record under `deployments/base-sepolia.json`
- recorded coverage report under `reports/coverage.md`

2.3 Files and Concerns Out of Scope

- frontend business logic and UX correctness beyond buildability
- hosted subgraph deployment and indexing infrastructure operations
- OpenZeppelin and Chainlink third-party library internals under `lib/`
- `node\_modules/` and package manager security review
- operational wallet custody and signing-device hygiene
- L2 bridge security, sequencer risk, or RPC provider trust beyond high-level dependency analysis

Out-of-scope does not mean irrelevant. It means those topics were discussed architecturally, but not line-by-line audited as first-party code.

3. Methodology

The review combined manual analysis with repository-native validation.

3.1 Manual Review Approach

The code was read with emphasis on:

- privilege boundaries and owner/role-gated functions;
- state transitions around policy lifecycle changes;
- vault reserve accounting and payout ordering;
- oracle data validation and threshold semantics;
- upgradeability layout and governance handoff assumptions;
- deployment scripts and what they do or fail to prove operationally.

3.2 Tooling and Evidence Used

- ``forge build``
- ``forge test``
- ``forge coverage``
- manual reading of contract source and deployment scripts
- local Base Sepolia deployment validation and post-deployment role checks
- manual execution of ``VerifyDeployment`` after fixing a verification-script false negative

The reviewed branch had the following validated status at the time of documentation:

- ``forge test``: ``134 passed, 0 failed, 0 skipped``
- ``forge coverage``:
- lines: ``93.60% (760/812)``
- statements: ``92.67% (784/846)``
- branches: ``66.94% (83/124)``
- functions: ``88.44% (153/173)``

3.3 Slither Status

The repository includes ``reports/slither-output.md``, but the final branch does not yet include a fresh ``slither .`` output summary. Therefore:

- no "Slither clean" claim is made in this report;
- no tool-derived finding counts are fabricated here;
- the appendix records the current pending status honestly.

4. Findings Overview

ID	Title	Severity	Status	Location
AUD-01	Raw oracle thresholds can be misconfigured across feed decimal conventions	Medium	Acknowledged	<code>contracts/insurance/RiskRegistry.sol:15-23</code> , <code>contracts/insurance/ClaimManager.sol:63-68</code> , <code>contracts/insurance/ClaimManager.sol:97-102</code>
AUD-02	Timelock bootstrap admin revocation is not enforced by the deployment flow	Medium	Acknowledged	<code>script/Deploy.s.sol:148-163</code> , <code>script/VerifyDeployment.s.sol:45-53</code>
AUD-03	Upgradeable proxy governance handoff is not covered by the verification checklist	Low	Acknowledged	<code>contracts/upgrade/UpgradeableInsurancePoolV1.sol:26-35</code> , <code>contracts/upgrade/UpgradeableInsurancePoolV1.sol:49-53</code> , <code>script/Deploy.s.sol:165-176</code> , <code>script/VerifyDeployment.s.sol:17-53</code>
AUD-04	Verification script used an invalid current governance timepoint	Informational	Fixed	<code>script/VerifyDeployment.s.sol:33-37</code>
AUD-05	Core wiring and pause hooks remain highly privileged admin surfaces	Low	Acknowledged	<code>contracts/nft/PolicyNFT.sol:38-53</code> , <code>contracts/vault/InsuranceVault.sol:46-67</code> , <code>contracts/insurance/InsurancePool.sol:108-121</code> , <code>contracts/factory/InsuranceFactory.sol:21-47</code>
AUD-06	<code>getActivePolicies()</code> scales linearly and performs a double scan	Gas	Acknowledged	<code>contracts/insurance/InsurancePool.sol:211-233</code>

5. Detailed Findings

AUD-01: Raw oracle thresholds can be misconfigured across feed decimal conventions

- Severity: Medium
- Status: Acknowledged
- Location:
- `contracts/insurance/RiskRegistry.sol:15-23``
- `contracts/insurance/RiskRegistry.sol:56-76``
- `contracts/insurance/RiskRegistry.sol:88-120``
- `contracts/insurance/ClaimManager.sol:63-68``
- `contracts/insurance/ClaimManager.sol:97-102``

Description

`RiskRegistry.RiskType`` stores `triggerThreshold`` as a plain `uint256`` with no associated feed-decimal metadata or normalization rule. `ClaimManager`` later approves claims by comparing the raw Chainlink answer against that raw stored

threshold:

- `latestPrice = oracleAdapter.getLatestPrice(...)`
- `approved = latestPrice <= riskType.triggerThreshold`

This design is simple, but it assumes governance will always configure the threshold in the exact numeric domain returned by the selected feed.

Impact

If governance or a privileged operator configures the threshold using the wrong decimal convention, the claim rule can become dramatically wrong:

- claims may become always claimable;
- claims may become impossible to trigger;
- the system may appear correct in the frontend while storing dangerous raw values on-chain.

Because this can lead to incorrect payouts or denial of valid payouts, the issue is classified as Medium severity.

Proof of Concept

Suppose a Chainlink feed returns prices with 8 decimals, and the intended claim rule is "approve when price is below 0.97". The correct raw threshold should be ``97_000_000``. If governance mistakenly stores an 18-decimal normalized number such as ``9700000000000000000``, then the condition:

Code Block: text

```
latestPrice <= triggerThreshold
```

becomes true across a huge range of ordinary prices, potentially allowing immediate claims that were never intended.

The reverse mistake also exists: storing a threshold that is too small can block all claims.

Recommendation

- persist feed-decimal metadata with each risk type, or query and normalize against ``AggregatorV3Interface.decimals()``;
- add configuration-time validation helpers and tests for threshold units;
- document the unit convention explicitly in governance and deployment procedures.

Status

Acknowledged. The current repository documents the issue operationally, but the on-chain representation is not yet normalized.

AUD-02: Timelock bootstrap admin revocation is not enforced by the deployment flow

- Severity: Medium
- Status: Acknowledged
- Location:
- ``script/Deploy.s.sol:148-163``
- ``script/VerifyDeployment.s.sol:45-53``

Description

The deployment script creates the timelock with:

- no proposers at construction time;
- open executors (``address(0)``);
- ``admin = config.initialOwner``.

The verification script later checks that:

- the timelock has the expected minimum delay;
- the governor has ``PROPOSER_ROLE``;
- the timelock has ``RISK_MANAGER_ROLE`` in the registry.

However, the verification script does not check that the bootstrap admin has renounced or been stripped of ``DEFAULT_ADMIN_ROLE``. That means the operational runbook can appear "verified" while an external EOA or multisig still retains the ability to reconfigure timelock roles.

Impact

If bootstrap admin privileges persist, a compromised admin can undermine the governance boundary by:

- granting `PROPOSER\_ROLE` to an attacker-controlled address;
- granting or revoking other timelock roles;
- restoring privileged access after the team believes governance handoff is complete.

This does not automatically drain funds in one transaction, but it materially weakens the intended timelock security model, so the issue is Medium severity.

Proof of Concept

1. Deploy the timelock using the current script, which sets `admin = initialOwner`.
2. Complete the ownership transfers and governor proposer grant.
3. Omit `renounceRole(DEFAULT\_ADMIN\_ROLE, bootstrapAdmin)` or equivalent revocation.
4. The bootstrap admin can still call timelock `grantRole(PROPOSER\_ROLE, attacker)` and then schedule privileged calls through the attacker account.

Recommendation

- add an explicit post-setup step that revokes or renounces the bootstrap admin role;
- extend `VerifyDeployment` to assert that no unintended EOA retains `DEFAULT\_ADMIN\_ROLE`;
- document this as a release blocker, not an optional cleanup task.

Status

Acknowledged. The current deployment documentation does not prove that admin revocation occurred.

AUD-03: Upgradeable proxy governance handoff is not covered by the verification checklist

- Severity: Low
- Status: Acknowledged
- Location:

- ``contracts/upgrade/UpgradeableInsurancePoolV1.sol:26-35``
- ``contracts/upgrade/UpgradeableInsurancePoolV1.sol:49-53``
- ``script/Deploy.s.sol:165-176``
- ``script/VerifyDeployment.s.sol:17-53``

Description

The UUPS example proxy is initialized with ``initialOwner``, and upgrades are authorized by ``onlyOwner`` through ``_authorizeUpgrade``. The verification script checks timelock ownership for the core NFT/vault/pool/factory contracts, but it does ****not**** check whether proxy ownership was transferred away from the bootstrap owner.

This matters because the proxy path is part of the repository's deployment story. Even if it is an isolated example rather than the live insurance core, its governance state should be clearly classified.

Impact

If proxy ownership remains with the bootstrap EOA or multisig:

- that actor can unilaterally upgrade the example proxy;
- the repository may appear more governance-complete than it really is;
- reviewers may overestimate how much of the deployment is timelock-controlled.

The severity is Low rather than Medium because the proxy example is not wired into the insured funds path, but the issue is still important for correctness of governance claims.

Proof of Concept

1. Deploy V1 and the proxy with ``initialize(initialOwner, collateralAsset)``.
2. Do not transfer proxy ownership to the timelock.
3. The bootstrap owner can still call ``upgradeToAndCall`` through the proxy and replace the implementation.

Recommendation

- explicitly transfer proxy ownership to the timelock if the example is meant

to be governed;

- or clearly mark the proxy example as a non-governed demonstration module;
- add proxy ownership assertions to `VerifyDeployment`.

Status

Acknowledged. The verification checklist still omits this proof.

AUD-04: Verification script used an invalid current governance timepoint

- Severity: Informational
- Status: Fixed
- Location:
- `script/VerifyDeployment.s.sol:33-37`

Description

The governance token uses timestamp-based checkpoints. During review, the verification script was found to query quorum at the current timepoint instead of a past one, which can trigger `ERC5805FutureLookup` and fail verification even when the deployment is otherwise correct.

The reviewed commit now uses `currentClock - 1`, which is the correct pattern for a checkpointed governance token.

Impact

This issue did not expose user funds. Its impact was false-negative deployment verification and operational confusion.

Proof of Concept

The pre-fix version queried:

Code Block: text

```
governor.quorum(currentClock)
```

With timestamp checkpoints, this can request a future lookup. The merged fix

uses:

Code Block: text

```
governor.quorum(currentClock - 1)
```

Recommendation

Keep all governance verification code timepoint-aware and prefer querying past checkpoints.

Status

Fixed in the current reviewed commit.

AUD-05: Core wiring and pause hooks remain highly privileged admin surfaces

- Severity: Low
- Status: Acknowledged
- Location:
- `contracts/nft/PolicyNFT.sol:38-53`
- `contracts/vault/InsuranceVault.sol:46-67`
- `contracts/insurance/InsurancePool.sol:108-121`
- `contracts/factory/InsuranceFactory.sol:21-47`

Description

Several contracts expose security-sensitive owner-only functions:

- `PolicyNFT.setInsurancePool`
- `PolicyNFT.setBaseTokenUri`
- `InsuranceVault.setInsurancePool`
- `InsuranceVault.setClaimManager`
- `InsuranceVault.pause/unpause`
- `InsurancePool.setClaimManager`
- `InsurancePool.pause/unpause`
- `InsuranceFactory.deployCreate/deployCreate2`

These powers are deliberate, but they create a broad control plane that must be fully transferred to governance before the protocol can be treated as decentralized in practice.

Impact

A malicious or compromised owner can:

- freeze deposits, withdrawals, or policy purchase;
- repoint wiring to attacker-controlled contracts;
- mutate NFT metadata;
- deploy arbitrary bytecode through the factory.

Because these actions are gated and expected to move under governance, the issue is Low severity rather than a higher direct exploit severity.

Proof of Concept

Examples of privileged actions available to the current owner:

- call ``PolicyNFT.setInsurancePool(maliciousPool)``;
- call ``InsuranceVault.setClaimManager(maliciousManager)``;
- call ``InsurancePool.pause()``;
- call ``InsuranceFactory.deployCreate(creationCode)``.

Recommendation

- treat full owner handoff as a release requirement;
- narrow mutable wiring where feasible;
- document the blast radius of each owner-controlled hook in governance docs.

Status

Acknowledged. These are intentional powers, but they remain security-sensitive.

AUD-06: ``getActivePolicies()`` scales linearly and performs a double scan

- Severity: Gas
- Status: Acknowledged
- Location:
- ``contracts/insurance/InsurancePool.sol:211-233``

Description

``getActivePolicies()`` first loops through all historical policy IDs to count

active entries and then loops again to materialize the output array. This is a reasonable convenience view for a small course-project state size, but it scales poorly as policy count grows.

Impact

- RPC responses may become slow or fail for large datasets;
- on-chain composability is poor because another contract calling this view would inherit its $O(n)$ scan cost and memory growth;
- indexers and frontends should prefer event-derived or paginated access.

There is no direct fund-loss impact, so the issue is categorized as Gas.

Proof of Concept

For N policies, the function:

1. reads every policy once to count active entries;
2. allocates an array of size `activeCount`;
3. reads every policy again to copy active entries.

The work therefore scales linearly with the full policy history, not with the number of active policies only.

Recommendation

- paginate policy reads;
- emit and index enough events for off-chain filtering;
- rely on the subgraph or dedicated read models for historical enumerations.

Status

Acknowledged. The project already includes subgraph assets, which is the right direction for solving this at the integration layer.

6. Centralization Analysis

6.1 Intended Steady-State Model

The intended governance story is:

1. `GovernanceToken` holders create and vote on proposals;

2. `ProtocolGovernor` queues successful proposals;
3. `ProtocolTimelock` delays and executes privileged calls;
4. owner-only functions and registry role operations are exercised through the timelock rather than through a private wallet.

This is a reasonable and standard control model for a course DeFi protocol.

6.2 Practical Centralization Points

In practice, centralization still depends on operational handoff quality.

Role or Actor	Powers	What could go wrong
Bootstrap admin / owner	wiring, pauses, possible proxy upgrades, timelock role management if retained	governance can be bypassed before or after launch if bootstrap powers persist
Timelock	execute every privileged protocol call after delay	compromise or malicious control can rewrite protocol behavior
<code>RISK_MANAGER_ROLE</code> holder	set feeds, thresholds, coverage caps, premium rates	bad configuration can overpay or underpay claims
Factory owner	deploy arbitrary bytecode	protocol namespace can host malicious or misleading modules
Proxy owner	authorize upgrades of example module	code evolution can happen outside governance if not transferred
Governance majority	control proposals and votes	whale or colluding holders can dominate policy changes

6.3 Timelock Powers

If the timelock is correctly configured and bootstrap roles are revoked, it can:

- pause or rewire the pool and vault;
- change NFT minting authority and metadata base URI;
- create, update, and deactivate risk types;
- deploy arbitrary contracts via the factory;
- if proxy ownership is also transferred, upgrade the UUPS example.

This makes the timelock both a strong safety boundary and a powerful centralization concentration point.

6.4 If a Role Holder Is Malicious or Compromised

- If the bootstrap owner is compromised before full handoff, the attacker may freeze or rewire the live system.
- If the timelock is compromised, the attacker can still only act through the timelock execution model, but that is enough to rewrite most privileged settings.
- If `RISK_MANAGER_ROLE` is compromised, the attacker can make oracle feeds or thresholds meaningless even without changing business logic code.
- If an operational multisig is used in any of these roles, compromise of the multisig is equivalent to compromise of the role itself.

7. Governance Attack Analysis

7.1 Flash-Loan Governance Attacks

Current defenses:

- vote power uses `ERC20Votes` checkpoints rather than balance reads at execution time;
- governance settings use explicit proposal thresholds and quorum;
- successful actions still pass through a two-day timelock.

Residual risk:

- if `IGOV` becomes borrowable in an external market and can be held at the relevant snapshot timepoint, a well-funded attacker may still influence a vote;
- the repository does not implement additional anti-borrow voting constraints.

Conclusion: partially mitigated by checkpointing and delay, but not eliminated in an ecosystem where borrowable governance power exists.

7.2 Whale Attacks

Current defenses:

- 4% quorum prevents trivially tiny participation;
- 1% proposal threshold prevents zero-stake proposal spam.

Residual risk:

- a concentrated token distribution can still dominate outcomes;
- this is more a treasury/distribution issue than a coding issue.

Conclusion: technically standard, economically dependent on distribution.

7.3 Proposal Spam

Current defenses:

- proposal threshold set to `initialSupply / 100`;
- timelock delay slows down execution throughput;

- governance actions are not instant owner writes.

Residual risk:

- a holder above threshold can still create nuisance proposals;
- off-chain review and social filtering remain important.

Conclusion: moderately mitigated, not fully prevented.

7.4 Timelock Bypass

As-designed bypass resistance:

- core privileged actions are intended to flow through timelock ownership and roles;
- `VerifyDeployment` proves some of the handoff path.

Residual bypass risk:

- retaining bootstrap admin on the timelock weakens the model;
- retaining proxy ownership outside the timelock leaves one upgrade path bypass-capable;
- incomplete owner transfers leave direct write paths outside governance.

Conclusion: the contracts support a delayed governance model, but deployment discipline determines whether that model is actually achieved.

8. Oracle Attack Analysis

8.1 Price Manipulation

The protocol uses Chainlink rather than local AMM spot prices for claim decisions. This is a strong baseline defense against direct pool-manipulation attacks. However, the protection only holds if:

- the chosen feed is actually the correct semantic source for the insured risk;
- governance does not configure the wrong feed;
- threshold units match feed decimals.

Conclusion: direct AMM manipulation risk is lower than with a local spot oracle, but feed-selection and unit-configuration risk remain.

8.2 Stale Price Risk

The oracle adapter explicitly rejects:

- zero or non-positive prices;
- incomplete rounds;
- rounds whose `updatedAt` exceeds the configured `stalenessLimit`.

This is a meaningful control and one of the stronger parts of the design.

Residual risk:

- governance still chooses the `stalenessLimit`;
- an overly loose limit can make data less meaningful even if technically "not stale" under the configured threshold.

8.3 Feed Depeg and Semantic Drift

Even a live and honest feed can be the wrong feed for the insured event. A depeg or price dislocation may also be the event users **want** to insure, rather than an oracle failure. The architecture therefore treats feed correctness as a business-governance question, not only a technical one.

Conclusion: feed depeg is not automatically a bug. It is part of the insured domain, and the protocol must set thresholds with that in mind.

8.4 L2 and External Dependency Note

Because the system runs on Base Sepolia for the recorded deployment:

- RPC outages can degrade frontend/subgraph usability;
- Chainlink feed availability is an external dependency;
- the protocol's code cannot defend against all external-network assumptions.

This is not a direct contract vulnerability, but it belongs in the threat model.

9. Slither Appendix

Appendix A: Slither Rerun Summary

A filtered Slither rerun was executed against the final reviewed branch using the project configuration file.

Command used:

```
slither . --config-file ../slither.config.json --json slither-results-clean.json
```

The rerun focused on the project contracts and excluded dependency, test, and script paths through the configured filters.

Summary of reported findings:

- Critical: 0
- High: 0
- Medium: 0
- Low: 5
- Informational: 4

The Low-severity findings are timestamp-related review flags in:

- contracts/insurance/ClaimManager.sol
- contracts/insurance/InsurancePool.sol
- contracts/oracle/ChainlinkOracleAdapter.sol

The Informational findings are:

- inline assembly usage in contracts/factory/InsuranceFactory.sol
- inline assembly usage in contracts/benchmark/AssemblyBenchmark.sol
- naming convention warning for CLOCK_MODE() in contracts/token/GovernanceToken.sol
- naming convention warning for __gap in contracts/upgrade/UpgradeableInsurancePoolV1.sol

No Critical, High, or Medium findings were reported in the filtered rerun.

Interpretation:

- the timestamp findings correspond to expected time-based business logic and stale-price protection checks, but are still surfaced by Slither as low-severity review items;
- the assembly findings are limited to a controlled deployment helper and the benchmark contract;
- the naming-convention findings are informational only and do not affect runtime safety.

The detailed rerun output is recorded in:

- reports/slither-output.md
- slither-results-clean.json

10. Final Assessment

The reviewed codebase is strong for a university final project:

- the core insurance logic is modular;

- reserve-aware collateral logic is implemented;
- claim execution is separated from policy issuance;
- governance and timelock primitives are present and demonstrably usable;
- the code is backed by a substantial automated test suite.

The most important remaining security work is not "rewrite the protocol from scratch" but:

1. tighten deployment and governance handoff procedures;
2. normalize or validate oracle threshold units more defensively;
3. complete and attach a fresh Slither rerun;
4. keep documentation explicit about what is deployed, what is verified, and what is still only partially operational.

With those caveats stated honestly, the project is in a credible and well-structured state for academic demonstration.